

Hash Tables

A hash table is a data structure that implements a dictionary: it supports fast SEARCH, INSERT, and DELETE operations by mapping keys to values using a hash function.

Natural Explanation

Instead of searching through a list (slow), hash a key to get an index into an array (fast). The hash function converts any key into a valid array position.

Ideal case: hash distributes keys evenly across array positions, so all operations are $O(1)$ on average.

Core Idea

Store key-value pairs in an array of size m : - **Hash function** $h : \text{key} \rightarrow [0, m-1]$ maps each key to an array index - Store pair at position $h(\text{key})$ - Retrieve, update, or delete by hashing the key and accessing that position

Core Operations

INSERT(key, value): - Compute $i = h(\text{key})$ - Store or update the pair at position i - Handle collision if another key also hashes to i

SEARCH(key): - Compute $i = h(\text{key})$ - Look up position i - Return value if found, null otherwise

DELETE(key): - Compute $i = h(\text{key})$ - Remove the pair at position i (mark or rearrange)

Load Factor

The **load factor** $\alpha = \frac{n}{m}$ is the ratio of elements to table size.

- If α is small, collisions are rare, and operations are $O(1)$ average
 - If α grows too large, collisions increase, degrading performance
 - Good practice: rebuild (resize) the table when α exceeds a threshold (often 0.75)
-

Handling Collisions

When two keys hash to the same position, a collision occurs. Two main strategies:

[[Collision Handling]]: chaining and linear probing are the standard techniques.

Requirements for Good Hash Function

- **Deterministic**: same input always gives same output
- **Uniform distribution**: keys should hash to all positions roughly equally
- **Fast to compute**: $O(1)$ time
- **Low collision rate**: minimal clustering

See [[Hash Functions]] for strategies.

Average vs Worst Case

- **Average case** (good hash, low α): SEARCH/INSERT/DELETE are $O(1)$
- **Worst case** (all collisions): operations degrade to $O(n)$

Well-implemented hash tables with good hash functions operate near average case in practice.

Use Cases

- Dictionary/map data structure in modern languages
 - Caching and memoization
 - Storing unique elements (like sets)
 - Quick membership testing
-

Related

- [[Hash Functions]]
- [[Collision Handling]]
- [[Running Time]]